

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL
CUSCO

CARRERA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE
SISTEMAS

GUÍA DE LABORATORIO N.º 01

Complejidad y experimentación algorítmica

Asignatura:	Algoritmos Avanzados
Unidad:	I. Complejidad, algoritmos de aproximación y Montículos de Fibonacci
Semestre académico:	2026-II
Duración:	120 minutos
Organización:	Trabajo grupal en los equipos del proyecto semestral
Integrantes:	Cuatro estudiantes por equipo
Lenguaje sugerido:	Python 3
Docente:	Héctor Eduardo Ugarte Rojas

Equipo: _____

Integrantes: _____

Fecha: _____

Índice

1. Objetivo de la práctica	2
2. Trabajo preparatorio	2
3. Materiales	2
4. Marco teórico breve	3
4.1. Tamaño de entrada y complejidad	3
4.2. Experimentación reproducible	3
4.3. Contadores de operaciones	3
4.4. Estrategias exactas	3
5. Distribución de fiabilidad del tiempo	4
6. Ejercicios resueltos	4
6.1. Ejercicio resuelto 1: crecimiento lineal y cuadrático	4
6.2. Ejercicio resuelto 2: búsqueda lineal y binaria	5
6.3. Ejercicio resuelto 3: Subset Sum mediante tres estrategias	6
7. Ejercicios propuestos	8
7.1. Ejercicio propuesto 1: Insertion Sort frente a Merge Sort	8
7.2. Ejercicio propuesto 2: límite práctico del crecimiento exponencial	8
8. Organización del trabajo grupal	8
9. Producto de entrega	9
10. Tabla de evaluación	9

1. Objetivo de la práctica

Objetivo

Implementar, instrumentar y comparar algoritmos con diferentes órdenes de crecimiento, relacionando el análisis asintótico con el tiempo de ejecución observado y el número de operaciones realizadas. El equipo aplicará una metodología básica de experimentación reproducible y comparará fuerza bruta, Programación Dinámica y Ramificar-Podar sobre instancias pequeñas de un mismo problema.

Al finalizar la práctica, el equipo será capaz de:

1. medir tiempos sin incluir innecesariamente la generación de datos;
2. instrumentar algoritmos mediante contadores de operaciones significativas;
3. relacionar tendencias experimentales con funciones de complejidad;
4. comparar algoritmos bajo las mismas condiciones e instancias;
5. identificar las limitaciones de inferir complejidad solo mediante mediciones;
6. explicar individualmente el funcionamiento y los resultados del código grupal.

2. Trabajo preparatorio

Antes del laboratorio, cada integrante deberá:

1. revisar notación $\mathcal{O}$, tamaño de entrada y complejidad polinomial, pseudo-polinomial y exponencial;
2. recordar las ideas generales de Programación Dinámica y Ramificar-Podar;
3. verificar la instalación de Python 3 y del entorno de desarrollo;
4. revisar la documentación de `time.perf_counter`, `statistics.median` y `random.seed`;
5. responder brevemente:
 - a) ¿Por qué una sola medición puede ser poco confiable?
 - b) ¿Cuál es la diferencia entre el valor de un entero y su tamaño de representación?
 - c) ¿Por qué dos algoritmos con igual complejidad asintótica pueden mostrar tiempos distintos?

Preparación grupal

Los integrantes pueden coordinar la preparación, pero cada estudiante debe comprender los conceptos, el código y las decisiones de experimentación. El docente podrá seleccionar a cualquier integrante para explicar o modificar una sección.

3. Materiales

- Computadora con Windows, GNU/Linux o macOS.
- Python 3.11 o posterior, recomendado.

- Visual Studio Code, PyCharm o IDE equivalente.
- Terminal o consola.
- Biblioteca estándar: `time`, `random`, `statistics` y `csv`.
- Git y repositorio del equipo.
- Jupyter Notebook o una hoja de cálculo, opcional para organizar resultados.
- Biblioteca de gráficos, opcional para representar las mediciones.

4. Marco teórico breve

4.1. Tamaño de entrada y complejidad

El costo de un algoritmo se expresa en función del tamaño de su entrada. Para una lista, n suele representar la cantidad de elementos. Para un entero positivo N codificado en binario, su tamaño es $\Theta(\log N)$ bits. Esta diferencia permite reconocer algoritmos pseudo-polinomiales, cuyo costo es polinomial respecto del valor de un parámetro, pero no necesariamente respecto de la longitud de su representación.

Entre los órdenes de crecimiento más frecuentes se encuentran:

$$\mathcal{O}(1), \quad \mathcal{O}(\log n), \quad \mathcal{O}(n), \quad \mathcal{O}(n \log n), \quad \mathcal{O}(n^2), \quad \mathcal{O}(2^n), \quad \mathcal{O}(n!).$$

4.2. Experimentación reproducible

El tiempo observado depende del equipo, sistema operativo, intérprete, memoria y procesos concurrentes. Una comparación básica debe usar iguales instancias, separar la preparación de datos del intervalo medido, repetir las ejecuciones, reportar una medida representativa y conservar los datos o semillas utilizados.

4.3. Contadores de operaciones

Los contadores complementan la medición temporal. La operación contada debe definirse claramente: comparación de elementos, candidato evaluado, llamada recursiva, nodo explorado o estado procesado. Métricas diferentes describen trabajo interno distinto y no deben tratarse como si fueran la misma unidad.

4.4. Estrategias exactas

La fuerza bruta examina todas o muchas soluciones candidatas. La Programación Dinámica almacena resultados de subproblemas para evitar recomputación. Ramificar-Podar recorre un árbol de decisiones y descarta ramas que no pueden producir una solución válida o mejorar la solución conocida. Las tres estrategias pueden ser exactas, pero presentan comportamientos diferentes según la instancia y sus parámetros.

5. Distribución de fiabilidad del tiempo

Etapa	Actividad	Tiempo
Inicio	Organización del equipo y revisión del trabajo preparatorio	10 min
Resuelto 1	Crecimiento lineal y cuadrático	18 min
Resuelto 2	Búsqueda lineal y binaria instrumentadas	18 min
Resuelto 3	Subset Sum mediante tres estrategias	24 min
Propuesto 1	Comparación de algoritmos de ordenamiento	20 min
Propuesto 2	Límite práctico del crecimiento exponencial	20 min
Cierre	Revisión individual, resultados y organización de entrega	10 min

6. Ejercicios resueltos

6.1. Ejercicio resuelto 1: crecimiento lineal y cuadrático

Enunciado. Implementar un algoritmo que sume una lista y otro que examine todos los pares de índices $i < j$. Medir el tiempo para tamaños crecientes.

El primer algoritmo recorre n elementos, por lo que tiene costo $\mathcal{O}(n)$. El segundo examina $n(n-1)/2$ pares, por lo que tiene costo $\Theta(n^2)$.

Listing 1: Medición de algoritmos lineal y cuadrático

```

1 from random import Random
2 from statistics import median
3 from time import perf_counter
4
5
6 def suma_lineal(datos):
7     total = 0
8     for valor in datos:
9         total += valor
10    return total
11
12
13 def contar_pares_ordenados(datos):
14     contador = 0
15     n = len(datos)
16     for i in range(n):
17         for j in range(i + 1, n):
18             if datos[i] <= datos[j]:
19                 contador += 1
20    return contador
21
22
23 def medir_mediana(funcion, datos, repeticiones=7):
24     tiempos = []
25     resultado = None
26     for _ in range(repeticiones):
27         inicio = perf_counter()
28         resultado = funcion(datos)
29         tiempos.append(perf_counter() - inicio)
30    return resultado, median(tiempos)
31
32

```

```
33 generador = Random(2026)
34 for n in [100, 500, 1000, 2000, 4000]:
35     datos = [generador.randint(1, 10_000) for _ in range(n)]
36     _, t1 = medir_mediana(suma_lineal, datos)
37     _, t2 = medir_mediana(contar_pares_ordenados, datos)
38     print(n, t1, t2)
```

Interpretación esperada. Al duplicar n , el tiempo lineal tenderá a multiplicarse aproximadamente por 2 y el cuadrático aproximadamente por 4, siempre que los tamaños sean suficientes para observar la tendencia. Los factores exactos pueden variar.

Verificación

La generación de datos ocurre antes de iniciar el cronómetro. Los algoritmos comparados deben procesar la misma instancia.

6.2. Ejercicio resuelto 2: búsqueda lineal y binaria

Enunciado. Comparar búsqueda lineal y búsqueda binaria mediante un contador de comparaciones. La búsqueda binaria requiere una lista ordenada.

Listing 2: Búsquedas instrumentadas

```
1 def busqueda_lineal(datos, objetivo):
2     comparaciones = 0
3     for indice, valor in enumerate(datos):
4         comparaciones += 1
5         if valor == objetivo:
6             return indice, comparaciones
7     return -1, comparaciones
8
9
10 def busqueda_binaria(datos, objetivo):
11     izquierda = 0
12     derecha = len(datos) - 1
13     comparaciones = 0
14
15     while izquierda <= derecha:
16         medio = (izquierda + derecha) // 2
17         comparaciones += 1
18         if datos[medio] == objetivo:
19             return medio, comparaciones
20         if datos[medio] < objetivo:
21             izquierda = medio + 1
22         else:
23             derecha = medio - 1
24
25     return -1, comparaciones
26
27
28 for n in [16, 32, 64, 128, 256, 512, 1024]:
29     datos = list(range(n))
30     objetivo = n + 1
31     _, lineales = busqueda_lineal(datos, objetivo)
32     _, binarias = busqueda_binaria(datos, objetivo)
33     print(n, lineales, binarias)
```

Para un objetivo ausente, la búsqueda lineal realiza n comparaciones. La búsqueda binaria realiza

aproximadamente $\lfloor \log_2 n \rfloor + 1$. Con $n = 1024$, se esperan 1024 comparaciones lineales y cerca de 11 comparaciones binarias.

Costo del ordenamiento

Si la colección no estaba ordenada y solo se efectuará una búsqueda, una comparación completa debe considerar también el costo de ordenar. El contexto determina qué algoritmo es más pertinente.

6.3. Ejercicio resuelto 3: Subset Sum mediante tres estrategias

Enunciado. Dado un conjunto de enteros positivos y un objetivo T , determinar si existe un subconjunto cuya suma sea exactamente T :

$$\sum_{i=1}^n a_i x_i = T, \quad x_i \in \{0, 1\}.$$

a) Fuerza bruta

Listing 3: Subset Sum por fuerza bruta

```
1 def subset_sum_fuerza_bruta(valores, objetivo):
2     n = len(valores)
3     candidatos = 0
4
5     for mascara in range(1 << n):
6         candidatos += 1
7         suma = 0
8         seleccion = []
9         for i in range(n):
10             if mascara & (1 << i):
11                 suma += valores[i]
12                 seleccion.append(valores[i])
13         if suma == objetivo:
14             return True, seleccion, candidatos
15     return False, [], candidatos
```

b) Programación Dinámica

Listing 4: Subset Sum por Programación Dinámica

```
1 def subset_sum_dp(valores, objetivo):
2     alcanzable = {0: []}
3     estados = 0
4
5     for valor in valores:
6         nuevos = dict(alcanzable)
7         for suma, seleccion in alcanzable.items():
8             estados += 1
9             nueva_suma = suma + valor
10            if nueva_suma <= objetivo and nueva_suma not in nuevos:
11                nuevos[nueva_suma] = seleccion + [valor]
12        alcanzable = nuevos
13        if objetivo in alcanzable:
14            return True, alcanzable[objetivo], estados
15
16     return False, [], estados
```

c) Ramificar-Podar

Listing 5: Subset Sum mediante Ramificar-Podar

```
1 def subset_sum_ramificar_podar(valores, objetivo):
2     valores = sorted(valores, reverse=True)
3     n = len(valores)
4     restante = [0] * (n + 1)
5
6     for i in range(n - 1, -1, -1):
7         restante[i] = restante[i + 1] + valores[i]
8
9     nodos = 0
10
11     def buscar(i, suma, seleccion):
12         nonlocal nodos
13         nodos += 1
14
15         if suma == objetivo:
16             return seleccion.copy()
17         if i == n or suma > objetivo:
18             return None
19         if suma + restante[i] < objetivo:
20             return None
21
22         seleccion.append(valores[i])
23         solucion = buscar(i + 1, suma + valores[i], seleccion)
24         if solucion is not None:
25             return solucion
26         seleccion.pop()
27
28         return buscar(i + 1, suma, seleccion)
29
30     solucion = buscar(0, 0, [])
31     return solucion is not None, solucion or [], nodos
```

d) Prueba común

Listing 6: Ejecución comparativa

```
1 valores = [3, 7, 11, 13, 17, 19, 23, 29]
2 objetivo = 42
3
4 for algoritmo in [subset_sum_fuerza_bruta,
5                 subset_sum_dp,
6                 subset_sum_ramificar_podar]:
7     existe, seleccion, trabajo = algoritmo(valores, objetivo)
8     assert not existe or sum(seleccion) == objetivo
9     print(algoritmo.__name__, existe, seleccion, trabajo)
```

Interpretación. Fuerza bruta cuenta candidatos, Programación Dinámica cuenta estados y Ramificar-Podar cuenta nodos. Las métricas describen el trabajo interno de cada técnica, pero no son exactamente la misma operación elemental.

Supuesto del algoritmo

La poda supone enteros positivos. Si se permiten valores negativos, las condiciones `suma > objetivo` y la cota mediante la suma restante no son válidas en general.

7. Ejercicios propuestos

7.1. Ejercicio propuesto 1: Insertion Sort frente a Merge Sort

Implemente y compare **Insertion Sort** y **Merge Sort** utilizando listas de tamaños crecientes y tres tipos de entrada: aleatoria, ordenada y ordenada en sentido inverso.

Requisitos:

1. contar comparaciones entre elementos;
2. medir la mediana de al menos siete ejecuciones;
3. usar las mismas listas para ambos algoritmos;
4. comprobar que las salidas coincidan con `sorted(datos)`;
5. presentar una tabla con tamaño, tipo de entrada, comparaciones y tiempo;
6. generar al menos un gráfico de tiempo y uno de comparaciones;
7. explicar por qué el comportamiento de Insertion Sort depende del orden inicial.

Pregunta de análisis. ¿A partir de qué tamaño Merge Sort supera consistentemente a Insertion Sort en el equipo utilizado? La respuesta debe sustentarse en los datos y no presentarse como una constante universal.

7.2. Ejercicio propuesto 2: límite práctico del crecimiento exponencial

Amplíe Subset Sum generando instancias reproducibles con n creciente. Determine hasta qué tamaño puede ejecutarse la fuerza bruta dentro de un límite de tiempo razonable, y compare ese límite con Programación Dinámica y Ramificar-Podar.

Requisitos:

1. usar una semilla fija y conservar las instancias;
2. evaluar casos con solución y sin solución;
3. evitar bloquear el equipo mediante límites seguros de crecimiento;
4. registrar tiempo, resultado y métrica interna de trabajo;
5. incluir un caso favorable y otro desfavorable para la poda;
6. explicar por qué Ramificar-Podar depende de las cotas y del orden de exploración;
7. relacionar el crecimiento observado con el espacio de 2^n subconjuntos.

8. Organización del trabajo grupal

El equipo puede distribuir tareas operativas, pero todos los integrantes deben comprender los ejercicios y los resultados. Se sugiere rotar durante la práctica las siguientes responsabilidades:

- implementación e instrumentación;
- diseño y ejecución de pruebas;

- registro y organización de resultados;
- revisión de complejidad y preparación de la explicación.

La distribución no significa que cada integrante conozca únicamente una parte. El docente podrá seleccionar a cualquier integrante para ejecutar una prueba, explicar una función, predecir una salida o realizar una modificación breve.

9. Producto de entrega

Producto de entrega

El equipo entregará:

1. código fuente organizado y ejecutable (tanto ejercicios resueltos como propuestos);
2. informe de practica de laboratorio conteniendo:
3. integrantes e instrucciones de ejecución;
4. respuestas del trabajo preparatorio;
5. datos e instancias utilizados;
6. resultados de los ejercicios propuestos;
7. al menos un gráfico de tiempo y uno de operaciones o trabajo interno;
8. respuestas a las preguntas de análisis;
9. conclusiones grupales de 150 a 250 palabras.

10. Tabla de evaluación

Criterio	Indicador de logro	Puntaje
Asistencia y participación en el laboratorio	Asiste a la sesión, permanece durante el desarrollo de la práctica y participa activamente en las actividades, pruebas y revisiones solicitadas. Este criterio se determina individualmente.	4
Corrección y funcionamiento	Los ejercicios resueltos y los dos ejercicios propuestos están completos, se ejecutan correctamente, producen resultados válidos y consideran los casos solicitados. La implementación conserva la lógica de los algoritmos y presenta una organización adecuada.	8
Análisis de complejidad y resultados	El equipo determina y justifica la complejidad, interpreta las mediciones y relaciona los resultados experimentales con el comportamiento teórico, reconociendo limitaciones y casos adversos.	3

Criterio	Indicador de logro	Puntaje
Dominio individual	El estudiante explica el código y los algoritmos, responde preguntas técnicas, predice resultados y realiza una modificación breve o resuelve un caso adicional solicitado por el docente.	4
Total		19

Referencias

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to Algorithms* (4.^a ed.). MIT Press.
- [2] Kleinberg, J. y Tardos, É. (2005). *Algorithm Design*. Pearson/Addison-Wesley.
- [3] Dasgupta, S., Papadimitriou, C. H. y Vazirani, U. V. (2008). *Algorithms*. McGraw-Hill.
- [4] Massachusetts Institute of Technology. *Advanced Algorithms*. MIT OpenCourseWare. <https://ocw.mit.edu/courses/6-854j-advanced-algorithms-fall-2005/>
- [5] Python Software Foundation. *Python 3 Documentation*. <https://docs.python.org/3/>